LỜI CẢM ƠN

Nhóm chúng em xin gửi lời cảm ơn chân thành tới Đại học Công nghiệp Hà Nội và Trường Công nghệ Thông tin và Truyền thông đã tạo điều kiện để chúng em học tập và thực hiện đề tài này.

Chúng em xin cảm ơn các thầy cô đã giảng dạy học phần Deep Learning, cung cấp nền tảng về đại số tuyến tính, giải tích, chain rule và các phương pháp huấn luyện mạng nơ-ron. Đặc biệt, chúng em xin bày tỏ lòng biết ơn sâu sắc tới thầy cô hướng dẫn đã góp ý về cách đặt câu hỏi nghiên cứu, cách thiết kế thí nghiệm kiểm soát biến và cách trình bày kết quả một cách trung thực. Những nhận xét của thầy cô giúp chúng em phân biệt rõ đâu là kết quả đo được và đâu là suy luận lý thuyết, đồng thời tránh khái quát hoá vượt quá phạm vi thí nghiệm.

Trong quá trình thực hiện, nhóm đã cùng nhau tìm hiểu tài liệu về backpropagation và khởi tạo tham số, cài đặt backpropagation thủ công bằng NumPy, kiểm tra kết quả bằng PyTorch autograd và sai phân hữu hạn, chạy các lưới thí nghiệm trên Fashion-MNIST và tổng hợp số liệu. Mỗi thành viên đều tham gia vào các khâu lý thuyết, cài đặt, thí nghiệm và viết báo cáo.

Do thời gian và kinh nghiệm nghiên cứu còn hạn chế, báo cáo chắc chắn không tránh khỏi thiếu sót. Nhóm chúng em rất mong nhận được những góp ý của thầy cô và bạn bè để đề tài được hoàn thiện hơn trong thời gian tới.

Chúng em xin chân thành cảm ơn!

Nhóm 4 thực hiện

LỜI MỞ ĐẦU

Huấn luyện một mạng nơ-ron về bản chất là giải một bài toán tối ưu: tìm bộ tham số θ (các ma trận trọng số W và vector bias b) sao cho hàm mất mát $L(\theta)$ nhỏ nhất có thể. Để giải bài toán này bằng Gradient Descent, có hai câu hỏi bắt buộc phải trả lời trước khi nói đến việc "huấn luyện":

1. **Tính gradient thế nào?** Một mạng thực tế có thể có hàng triệu tham số; tính $\partial L/\partial \theta_i$ cho từng tham số một cách ngây thơ (ví dụ bằng finite-difference) là không khả thi. Đây là vấn đề mà **Backpropagation** giải quyết.
2. **Bắt đầu từ đâu?** Gradient Descent chỉ di chuyển tham số theo hướng giảm loss *cục bộ*, xuất phát từ giá trị khởi tạo θ_0. Chọn θ_0 sai có thể khiến mạng không bao giờ học được gì, bất kể thuật toán tối ưu tốt đến đâu. Đây là vấn đề của **Parameter Initialization**.

Báo cáo này đi từ nền tảng toán học (đạo hàm, chain rule, computational graph) tới hai chủ đề trên, xen kẽ trực giác trước rồi mới tới công thức và ví dụ số, tránh trình bày Backpropagation như một "thuật toán thần kỳ" tách rời khỏi Giải tích. Phần thực nghiệm (Mục 2.1–3.1) kiểm chứng lại các kết luận lý thuyết bằng số liệu thật trên Fashion-MNIST, không dùng số liệu minh hoạ giả định.

Sơ đồ tư duy của cả báo cáo. Chain Rule $\rightarrow$ Computational Graph $\rightarrow$ đạo hàm cục bộ từng bước $\rightarrow$ Backpropagation (thuật toán *tính* gradient hiệu quả) $\rightarrow$ Gradient Descent/Optimizer (*dùng* gradient đó để cập nhật tham số) — và song song: Initialization quyết định gradient có "chảy" được qua mạng sâu hay không (Mục 1.5), nên hai chủ đề của báo cáo thực chất là hai mặt của cùng một câu hỏi: *làm sao tín hiệu học (gradient) đi xuyên suốt một mạng nhiều lớp mà không biến mất hay bùng nổ?*

DANH MỤC TỪ VIẾT TẮT

Viết tắt	Giải thích
MLP	Multi-Layer Perceptron
ReLU	Rectified Linear Unit
SGD	Stochastic Gradient Descent
RMS	Root Mean Square
VJP	Vector–Jacobian Product
NaN	Not a Number
GD	Gradient Descent
LR	Learning Rate (tốc độ học η)
BN	Batch Normalization
LN	Layer Normalization
CE	Cross-Entropy (hàm mất mát entropy chéo)
MNIST	Modified National Institute of Standards and Technology (tập ảnh chữ số)
FMNIST	Fashion-MNIST (tập ảnh thời trang, 10 lớp)
CPU	Central Processing Unit
GPU	Graphics Processing Unit
Var	Variance (phương sai)
$E[a^2]$	Second moment của activation
fan_in / fan_out	Số đầu vào / đầu ra của một lớp
autograd	Cơ chế lấy đạo hàm tự động của PyTorch
Adam	Adaptive Moment Estimation (bộ tối ưu)
Momentum	Bộ tối ưu có quán tính
API	Application Programming Interface

DANH MỤC HÌNH ẢNH

1.1 Computational graph của $z = wx+b \to a = \text{ReLU}(z) \to L = \frac{1}{2}(a-y)^2$. Mũi tên xanh (liền) = forward; mũi tên đỏ (đứt) = backward, đi ngược đúng theo các cạnh forward. 11

1.2 Gradient đi ngược Loss → Output → Hidden → Input. Mỗi $\delta_l = \partial L/\partial z_l$ được tính một lần rồi dùng lại cho mọi tham số của lớp l — đây là điểm làm Backprop hiệu quả (Mục **??**). 14

1.3 Phân phối $10\,000$ giá trị W mô phỏng ngay sau khởi tạo theo từng scheme (lớp 128×128, khớp $n_{\text{in}}{=}n_{\text{out}}{=}128$ của kiến trúc thực nghiệm Mục 2.1) — Zero co về một điểm; độ rộng phân phối tăng dần Random $<$ LeCun $=$ Xavier (khi $n_{\text{in}}{=}n_{\text{out}}$) $<$ He, đúng thứ tự Var(W) ở Bảng 1.6. 20

1.4 Gradient RMS lớp 1 tại khởi tạo (trái) và test accuracy sau 8 epoch (phải), theo độ sâu mạng, activation ReLU. Đường chấm ở đồ thị phải = mức ngẫu nhiên (10%). 23

3.1 Test accuracy, toàn bộ lưới 5×4 (đường chấm = mức ngẫu nhiên 10%). 27

DANH MỤC BẢNG BIỂU

1.1 Forward-mode (JVP) vs. Reverse-mode (VJP) automatic differentiation 10

1.2 Forward và backward cho ví dụ số Mục 1.1.4 13

1.3 Forward/backward mạng 2 lớp — khớp PyTorch autograd (sai số $= 0$) và finite-difference (sai số $< 10^{-9}$) 15

1.4 Phân biệt Backpropagation và Gradient Descent 17

1.5 Truy vết 2 bước Gradient Descent với Zero-init — $W^{(1)}, W^{(2)}$ đứng yên tuyệt đối 18

1.6 Các scheme initialization dùng trong báo cáo (và trong lưới thực nghiệm Mục 3.1) 20

1.7 Minh hoạ số: hệ số nhân dồn g^L theo độ sâu L (đã tính lại bằng `experiments/toy_example_verify.py`) 22

2.1 Kết quả Gradient Checking (mạng $6 \to 8 \to 5 \to 1$, activation Tanh) . . 26

3.1 Test accuracy sau 15 epoch, toàn bộ 20 cấu hình 27

Mục lục

LỜI CẢM ƠN **1**

DANH MỤC TỪ VIẾT TẮT **3**

1 CƠ SỞ LÝ THUYẾT VÀ TOÁN HỌC **8**

1.1 Nền tảng toán học 8

1.1.1 Gradient 8

1.1.2 Jacobian — khi đầu ra không chỉ là một số 9

1.1.3 Quy tắc chuỗi (Chain Rule) 11

1.1.4 Đồ thị tính toán (Computational Graph) 11

1.2 Lan truyền tiến (Forward Propagation) 12

1.3 Lan truyền ngược (Backpropagation) 13

1.3.1 Scalar Derivation — ví dụ từ neuron đơn tới mạng 2 lớp 13

1.3.2 Matrix Form (batch, khớp code NumPy/PyTorch) 15

1.3.3 Backpropagation vs. Gradient Descent 17

1.4 Khởi tạo tham số (Parameter Initialization) 18

1.4.1 Vì sao Initialization quan trọng 18

1.4.2 Zero Initialization 18

1.4.3 Random Initialization 19

1.4.4 LeCun, Xavier/Glorot, He/Kaiming Initialization 19

1.5 Vanishing và Exploding Gradient 22

1.5.1 Thực nghiệm bổ sung: độ sâu ảnh hưởng thế nào tới initialization 23

2 THIẾT KẾ THỰC NGHIỆM **25**

2.1 Phương pháp thực nghiệm 25

2.1.1 Kiến trúc mô hình 25

2.2 Hiện thực 25
2.2.1 Part A: Manual Backpropagation 25
2.2.2 Kiểm tra tính đúng đắn: Gradient Checking 26

3 KẾT QUẢ VÀ THẢO LUẬN 27

3.1 Kết quả thực nghiệm 27
3.1.1 Bức tranh tổng thể: test accuracy 27
3.2 Thảo luận 28
3.2.1 Vì sao Sigmoid thất bại ngay cả với He/Xavier? 28
3.3 Hạn chế 29
Tài liệu tham khảo 30

Chương 1

CƠ SỞ LÝ THUYẾT VÀ TOÁN HỌC

1.1 Nền tảng toán học

Các khái niệm trong mục này là kiến thức Giải tích/Đại số tuyến tính chuẩn, trình bày lại theo hướng ứng dụng cho Deep Learning (tham khảo cách trình bày tổng quát trong Bishop [1] và Goodfellow et al. [2]).

1.1.1 Gradient

Gradient gộp toàn bộ đạo hàm riêng thành một vector — giống như gộp "độ dốc theo từng hướng riêng lẻ" thành một mũi tên chỉ hướng dốc nhất.

$$\nabla_\theta L = \left(\frac{\partial L}{\partial \theta_1}, \frac{\partial L}{\partial \theta_2}, \ldots, \frac{\partial L}{\partial \theta_n} \right).$$

$\nabla_\theta L$ là hướng làm L *tăng* nhanh nhất tại điểm hiện tại θ; do đó $-\nabla_\theta L$ là hướng làm L *giảm* nhanh nhất cục bộ — đây là nền tảng của Gradient Descent (Mục 1.3.3). Lưu ý: gradient tự nó chỉ là một *phép đo* (một vector), chưa phải là một thuật toán cập nhật tham số.

Vì sao chính xác là hướng dốc nhất? Với một hướng đơn vị bất kỳ v ($\|v\| = 1$), **đạo hàm theo hướng** (*directional derivative*) đo tốc độ đổi của L khi di chuyển theo v:

$$D_v L = \nabla_\theta L \cdot v = \|\nabla_\theta L\| \, \|v\| \cos\varphi = \|\nabla_\theta L\| \cos\varphi$$

(φ = góc giữa $\nabla_\theta L$ và v). Biểu thức này lớn nhất khi $\cos\varphi = 1$, tức v *cùng hướng* với $\nabla_\theta L$ ($\varphi = 0$) — chứng minh trực tiếp gradient là hướng tăng nhanh nhất; tương tự, $D_v L$ nhỏ nhất (âm nhiều nhất) khi v *ngược hướng* gradient ($\varphi = 180°$), tức hướng giảm nhanh nhất là $-\nabla_\theta L$.

Ví dụ số ($L(\theta_1, \theta_2) = \theta_1^2 + 2\theta_2^2$ tại $\theta = (1, 1)$): $\nabla L = (2\theta_1,\ 4\theta_2) = (2, 4)$, $\|\nabla L\| = \sqrt{20} \approx 4.47$. So sánh $D_v L$ theo 3 hướng đơn vị khác nhau:

- $v = \nabla L / \|\nabla L\| \approx (0.447;\ 0.894)$ (cùng hướng gradient): $D_v L = \sqrt{20} \approx 4.47$ (lớn nhất).

- $v = (1, 0)$ (chỉ đổi θ_1): $D_v L = (2, 4) \cdot (1, 0) = 2$.
- $v = -\nabla L / \|\nabla L\| \approx (-0.447;\ -0.894)$ (ngược hướng gradient): $D_v L = -\sqrt{20} \approx -4.47$ (nhỏ nhất — giảm nhanh nhất).

Đúng như dự đoán: đi theo $-\nabla L$ cho tốc độ giảm L nhanh hơn hẳn đi theo $(1, 0)$ hay bất kỳ hướng nào khác — đây chính xác là lý do Gradient Descent (Mục 1.3.3) chọn hướng cập nhật $-\nabla_\theta L$ chứ không phải hướng nào khác.

1.1.2 Jacobian — khi đầu ra không chỉ là một số

Gradient $\nabla_\theta L$ ở trên chỉ áp dụng khi đầu ra là *một số* (loss L luôn là một số). Nhưng bên trong mạng, nhiều phép biến đổi có đầu ra là *cả một vector* — ví dụ $z = Wx + b$ với $x \in \mathbb{R}^n$, $z \in \mathbb{R}^m$. Khi đó "đạo hàm" tổng quát thành một **ma trận Jacobian**: mỗi hàng là gradient của một toạ độ đầu ra theo toàn bộ đầu vào.

$$J = \frac{\partial z}{\partial x} = \begin{pmatrix} \dfrac{\partial z_1}{\partial x_1} & \cdots & \dfrac{\partial z_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \dfrac{\partial z_m}{\partial x_1} & \cdots & \dfrac{\partial z_m}{\partial x_n} \end{pmatrix} \in \mathbb{R}^{m \times n}.$$

Gradient là "1 số $\to$ nhiều số" (một hàng của Jacobian). Jacobian là "nhiều số $\to$ nhiều số" (toàn bộ ma trận). Backpropagation lan truyền *ngược* một vector "độ nhạy tích luỹ"qua từng Jacobian, không bao giờ lập ra Jacobian đầy đủ (rất tốn bộ nhớ với mạng lớn) — công thức hình thức ở dưới gọi đúng tên phép toán này: **Vector–Jacobian Product**.

Vector–Jacobian Product (VJP) và Reverse-mode AD. Xét một phép biến đổi $v_{i+1} = f_i(v_i)$ với Jacobian cục bộ $J_i = \partial v_{i+1} / \partial v_i$. Định nghĩa **cotangent** (hay *adjoint*) tại nút i là $\bar{v}_i \triangleq \partial L / \partial v_i$ (cùng shape với v_i, luôn xuất phát từ đạo hàm của *một số vô hướng* L — đúng trường hợp L là hàm mất mát). Quy tắc chuỗi cho cotangent là:

$$\bar{v}_i = \bar{v}_{i+1}\, J_i \qquad \Big(\text{viết theo vector cột: } \bar{v}_i^{\mathsf{T}} = J_i^{\mathsf{T}}\, \bar{v}_{i+1}^{\mathsf{T}}\Big).$$

Đây gọi là **Vector–Jacobian Product (VJP)**: nhân một *vector* ($\bar{v}_{i+1}$) với Jacobian J_i, đi *ngược* chiều forward — khác với *Jacobian–Vector Product (JVP)* của **forward-mode AD** (nhân Jacobian với một vector tangent $\dot{v}_i$, đi *xuôi* chiều forward: $\dot{v}_{i+1} = J_i \dot{v}_i$). Bảng 1.1 tóm tắt khác biệt.

Vì mạng nơ-ron luôn có *rất nhiều* tham số nhưng chỉ *một* loss vô hướng, reverse-

	Forward-mode (JVP)	**Reverse-mode (VJP) — Back-prop**
Lan truyền	Tangent $\dot{v}$ đi *xuôi* theo forward pass	Cotangent $\bar{v}$ đi *ngược* sau khi forward pass hoàn tất
Công thức	$\dot{v}_{i+1} = J_i \dot{v}_i$	$\bar{v}_i = \bar{v}_{i+1} J_i$
Chi phí cho $n_{\text{in}} \to 1$ đầu ra	$O(n_{\text{in}})$ lượt (1 lượt/hướng vào)	$O(1)$ lượt — đúng 1 lượt cho *mọi* tham số cùng lúc
Phù hợp khi	#outputs $\gg$ #inputs	#inputs $\gg$ #outputs (đúng trường hợp mạng nơ-ron: hàng triệu θ, 1 số L)

Bảng 1.1: Forward-mode (JVP) vs. Reverse-mode (VJP) automatic differentiation

mode (VJP) luôn được chọn — đây là lý do toán học chính xác cho lập luận "hiệu quả hơn theo bậc độ lớn"đã nêu ở Mục **??**, không chỉ là quan sát thực nghiệm.

Khớp chính xác với `torch.autograd`. PyTorch ghi lại một đồ thị tính toán (DAG) trong lúc forward: mỗi tensor kết quả giữ một con trỏ `grad_fn` trỏ tới một đối tượng `Function` biết cách tính VJP của chính phép toán đó. Lời gọi `loss.backward()` duyệt DAG này theo thứ tự tô-pô *ngược*, và tại mỗi nút gọi đúng phương thức

```
grad_input = Function.backward(ctx, grad_output)
```

— ở đây `grad_output` chính là $\bar{v}_{i+1}$ (cotangent nhận từ phía sau) và `grad_input` chính là $\bar{v}_i = \bar{v}_{i+1} J_i$ (VJP vừa định nghĩa). Nói cách khác, **mỗi hàm `backward()` trong PyTorch chính là lời giải tường minh của công thức VJP cho đúng một phép toán**; toàn bộ autograd engine chỉ là cơ chế ghép nối các lời giải cục bộ đó theo đúng thứ tự Chain Rule — không có "phép màu"nào khác.

Ví dụ: với $z = Wx + b$ ($W \in \mathbb{R}^{m \times n}$), Jacobian của z theo x chính là W: $\partial z / \partial x = W$, và VJP tương ứng $\bar{x} = \bar{z}W$ không bao giờ cần dựng W tường minh dưới dạng Jacobian — W *đã sẵn là* ma trận cần nhân. Với hàm kích hoạt $a = f(z)$ áp dụng *độc lập theo từng toạ độ* (mọi activation trong Mục 1.4.4 đều vậy), Jacobian của nó là một **ma trận đường chéo** $\text{diag}(f'(z_1), \ldots, f'(z_m))$, nên VJP $\bar{z} = \bar{a}\,\text{diag}(f'(z))$ rút gọn thành phép nhân từng phần tử $\bar{z} = \bar{a} \odot f'(z)$ — đây chính là lý do $\odot$ xuất hiện xuyên suốt Mục 1.3.1–1.3.2, thay vì một phép nhân ma trận đầy đủ tốn $O(m^2)$ bộ nhớ mỗi lớp.

1.1.3 Quy tắc chuỗi (Chain Rule)

$$y = f(g(x)) \quad \Longrightarrow \quad \frac{dy}{dx} = \frac{dy}{dg} \cdot \frac{dg}{dx}.$$

g biến đổi x thành một đại lượng trung gian; f biến đổi đại lượng đó thành y. Độ nhạy tổng của y theo x = độ nhạy của y theo đại lượng trung gian, nhân với độ nhạy của đại lượng trung gian theo x — "truyền độ nhạy" qua từng bước trung gian, giống truyền tin đồn qua một chuỗi người: mức độ "sai lệch" tới người cuối phụ thuộc vào mức khuếch đại/suy giảm ở *mỗi* khâu trung gian.

Một neuron đơn là một chuỗi biến đổi lồng nhau:

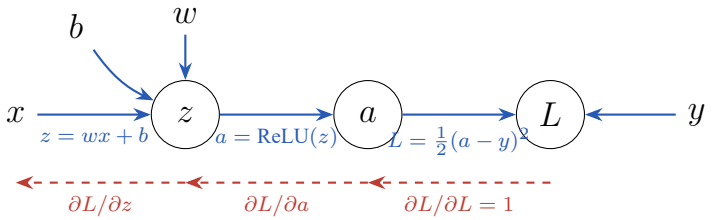
$$x \xrightarrow{z=wx+b} z \xrightarrow{a=\sigma(z)} a \xrightarrow{L=\text{loss}(a,y)} L.$$

Áp dụng Chain Rule liên tiếp cho từng tham số:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial w}, \qquad \frac{\partial L}{\partial b} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial b}, \qquad \frac{\partial L}{\partial x} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial x}.$$

$\partial L/\partial w$ và $\partial L/\partial b$ dùng để **cập nhật tham số của chính lớp này**; $\partial L/\partial x$ dùng để **tiếp tục lan truyền ngược sang lớp phía trước** (vì x chính là đầu ra của lớp trước đó) — đây là hạt giống của thuật toán Backpropagation ở Mục 1.3.

1.1.4 Đồ thị tính toán (Computational Graph)



Hình 1.1: Computational graph của $z = wx + b \to a = \text{ReLU}(z) \to L = \frac{1}{2}(a-y)^2$. Mũi tên xanh (liền) = forward; mũi tên đỏ (đứt) = backward, đi ngược đúng theo các cạnh forward.

Đạo hàm cục bộ của từng nút trong Hình 1.1:

$$\frac{\partial L}{\partial a} = a - y, \qquad \frac{\partial a}{\partial z} = \text{ReLU}'(z) = \begin{cases} 1, & z > 0 \\ 0, & z < 0 \end{cases}, \qquad \frac{\partial z}{\partial w} = x, \quad \frac{\partial z}{\partial b} = 1, \quad \frac{\partial z}{\partial x} = w.$$

Tại $z = 0$: dưới đạo hàm (subgradient). $\text{ReLU}(z) = \max(0, z)$ *không khả vi* tại

$z = 0$ (đồ thị có một "góc gãy", hai phía có độ dốc khác nhau: 0 bên trái, 1 bên phải) — đạo hàm thường không tồn tại tại điểm này. Vì ReLU là hàm **lồi** (*convex*), công cụ thay thế đúng đắn về giải tích là **tập dưới vi phân** (*subdifferential*):

$$\partial f(z_0) \triangleq \{\, c \in \mathbb{R} : f(z) \geq f(z_0) + c\,(z - z_0) \;\; \forall z \,\}$$

(tập mọi độ dốc c sao cho đường thẳng qua $(z_0, f(z_0))$ độ dốc c luôn nằm *dưới hoặc chạm* đồ thị f — với hàm khả vi thông thường, tập này chỉ có đúng 1 phần tử $= f'(z_0)$). Với ReLU tại $z_0 = 0$, kiểm tra trực tiếp định nghĩa cho:

$$\partial\,\mathrm{ReLU}(0) = [0, 1]$$

(mọi $c \in [0, 1]$ đều thoả bất đẳng thức trên; xem chứng minh nhanh: với $z \geq 0$ cần $z \geq cz \Leftrightarrow c \leq 1$; với $z < 0$ cần $0 \geq cz \Leftrightarrow c \geq 0$). Về mặt lý thuyết, **bất kỳ** giá trị nào trong $[0, 1]$ đều là một dưới đạo hàm hợp lệ tại $z = 0$.

Quy ước kỹ thuật của framework. Để có một con số *duy nhất* dùng trong backward pass (Mục 1.3.2 cần $f'(z)$ là một hàm số, không phải một tập), PyTorch và hầu hết framework chọn cố định

$$\left.\frac{\partial\,\mathrm{ReLU}}{\partial z}\right|_{z=0} = 0$$

— một lựa chọn *hợp lệ* (nằm trong $[0, 1]$) nhưng *không duy nhất về mặt toán học*. Lựa chọn này không gây vấn đề thực tế vì hai lý do: (1) với z liên tục (số thực dấu phẩy động), sự kiện z *đúng bằng* 0 tuyệt đối có xác suất gần như 0 (tập đo-không) — ngoại lệ đáng chú ý duy nhất trong báo cáo này là chính kịch bản Zero-init (Mục 1.4.2), nơi $z \equiv 0$ *cho mọi* neuron một cách có chủ đích; (2) với các thuật toán tối ưu dựa trên dưới đạo hàm (*subgradient method*), bất kỳ lựa chọn nào trong $[0, 1]$ đều cho một hướng giảm hợp lệ, nên chọn 0 (đơn giản nhất về mặt cài đặt, tương đương coi ReLU "chết"tại đúng điểm gãy) không ảnh hưởng tính đúng đắn của Gradient Descent trong thực hành.

Ví dụ số ($x = 2,\ w = 0.5,\ b = -0.3,\ y = 1$, đã kiểm chứng bằng NumPy, PyTorch autograd và finite-difference trong `experiments/toy_example_verify.py` — ba cách tính cho kết quả khớp tuyệt đối):

1.2 Lan truyền tiến (Forward Propagation)

Một MLP (*Multilayer Perceptron*) L lớp áp dụng liên tiếp phép biến đổi tuyến tính rồi phi tuyến. Ký hiệu $a^{(l)}$ là đầu ra (post-activation) của lớp l, $W^{(l)}, b^{(l)}$ là trọng số/bias của

Bảng 1.2: Forward và backward cho ví dụ số Mục 1.1.4

Forward	**Giá trị**	**Backward**	**Giá trị**
$z = wx + b$	0.7000	$\partial L/\partial a = a - y$	-0.3000
$a = \mathrm{ReLU}(z)$	0.7000	$\partial a/\partial z$ $(z > 0)$	1.0000
$L = \frac{1}{2}(a - y)^2$	0.0450	$\partial L/\partial z$	-0.3000
		$\partial L/\partial w = (\partial L/\partial z)\, x$	-0.6000
		$\partial L/\partial b$	-0.3000
		$\partial L/\partial x = (\partial L/\partial z)\, w$	-0.1500

lớp l, f là hàm kích hoạt:

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}, \qquad a^{(l)} = f\big(z^{(l)}\big), \qquad a^{(0)} = x.$$

Lớp cuối $a^{(L)} = \hat{y}$ là dự đoán của mạng; với bài toán phân loại, lớp cuối thường *không* qua f phi tuyến mà giữ logits thô, để hàm mất mát (vd. Cross-Entropy) tự áp dụng softmax bên trong (tránh tính softmax hai lần và giúp gradient ổn định hơn về số học).

Ví dụ nhỏ: mạng 1 lớp ẩn, $x \in \mathbb{R}^2$, $W^{(1)} \in \mathbb{R}^{3\times 2}$, $f = \mathrm{ReLU}$: mỗi trong 3 neuron ẩn tính $z_j^{(1)} = W_{j,:}^{(1)} \cdot x + b_j^{(1)}$ rồi qua ReLU độc lập với nhau — forward pass chỉ đơn giản là lặp lại "nhân ma trận $\rightarrow$ cộng bias $\rightarrow$ áp activation" qua từng lớp. Mục 1.3.2 viết lại đúng phép tính này ở dạng ma trận cho cả một batch.

1.3 Lan truyền ngược (Backpropagation)

1.3.1 Scalar Derivation — ví dụ từ neuron đơn tới mạng 2 lớp

Bước 1 — một neuron (đã làm ở Mục 1.1.4). Với đồ thị $z = wx+b \rightarrow a = \mathrm{ReLU}(z) \rightarrow L = \frac{1}{2}(a - y)^2$, ba đạo hàm cần cho huấn luyện là $\partial L/\partial w,\ \partial L/\partial b$ (cập nhật tham số) và $\partial L/\partial x$ (tiếp tục lan truyền ngược nếu có lớp trước đó) — xem Bảng 1.2.

Lưu ý đồng bộ ký hiệu. Mục 1.1.2 viết phép biến đổi tuyến tính tổng quát dạng $z = Wx+b$ với $W \in \mathbb{R}^{m\times n}$ nhân *bên trái* vector cột $x \in \mathbb{R}^n$ — quy ước đại số tuyến tính chuẩn cho một phép biến đổi $\mathbb{R}^n \rightarrow \mathbb{R}^m$. Từ đây trở đi, báo cáo chuyển sang quy ước phổ biến hơn trong tài liệu mạng nơ-ron: mỗi neuron có một vector trọng số riêng đặt theo *cột* của ma trận W (cột k của W là trọng số của neuron k), nên phép nhân tương ứng viết $z = W^\top x + b$ (tức W ở quy ước Mục 1.1.2 chính là $W^\top$ ở đây). Hai cách viết mô tả cùng một phép tính — khác nhau chỉ ở việc trọng số mỗi neuron được lưu theo hàng hay theo

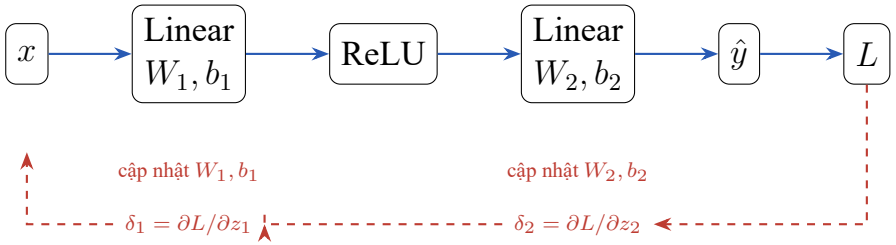
cột của ma trận — và *quy ước hàng-là-mẫu, cột-là-neuron* ($z = W^\mathsf{T}x + b$) là quy ước dùng xuyên suốt từ đây, khớp với cách NumPy/PyTorch lưu tham số (`weight.shape = (fan_out, fan_in)`).

Bước 2 — mạng 2 lớp. Xét kiến trúc (quy ước vector cột cho *một mẫu đơn lẻ* vừa nêu ở trên; Mục 1.3.2 sẽ đổi sang quy ước hàng-là-mẫu (batch) để khớp code):

$$x \to \text{Linear}(W_1, b_1) \to \text{ReLU} \to \text{Linear}(W_2, b_2) \to \hat{y} \to L,$$

với:

$$z_1 = W_1^\mathsf{T}x + b_1, \quad a_1 = \text{ReLU}(z_1), \quad z_2 = W_2^\mathsf{T}a_1 + b_2, \quad \hat{y} = z_2, \quad L = \tfrac{1}{2}\|\hat{y} - y\|^2.$$



Hình 1.2: Gradient đi ngược Loss → Output → Hidden → Input. Mỗi $\delta_l = \partial L/\partial z_l$ được tính một lần rồi dùng lại cho mọi tham số của lớp l — đây là điểm làm Backprop hiệu quả (Mục **??**).

Backward:

$$\delta_2 = \frac{\partial L}{\partial z_2} = \hat{y} - y \quad \text{(output activation = identity)}$$

$$\frac{\partial L}{\partial W_2} = a_1\,\delta_2^\mathsf{T}, \qquad \frac{\partial L}{\partial b_2} = \delta_2, \qquad \frac{\partial L}{\partial a_1} = W_2\,\delta_2$$

$$\delta_1 = \frac{\partial L}{\partial z_1} = \frac{\partial L}{\partial a_1} \odot \text{ReLU}'(z_1)$$

$$\frac{\partial L}{\partial W_1} = x\,\delta_1^\mathsf{T}, \qquad \frac{\partial L}{\partial b_1} = \delta_1, \qquad \frac{\partial L}{\partial x} = W_1\,\delta_1$$

($\odot$ = nhân từng phần tử. Vì $a = f(z)$ tác động độc lập lên từng toạ độ, Jacobian của nó là ma trận đường chéo, nên nhân Jacobian-vector rút gọn thành nhân từng phần tử với $f'(z)$.)

Vì sao có δ_1, δ_2 riêng, không viết lại Chain Rule từ đầu mỗi lần? Mỗi δ_l "gộp sẵn" toàn bộ ảnh hưởng của các lớp phía sau nó; khi tính gradient cho W_1, b_1 chỉ cần nhân δ_1 với input cục bộ (x) — không phải viết lại chuỗi nhân xuyên suốt từ L. Đây chính là điểm làm Backprop **hiệu quả**, không chỉ **đúng**.

Ví dụ số đã verify ($x = (1, 1)$, xem thiết lập tham số đầy đủ trong `experiments/toy_example_verify.py`):

Bảng 1.3: Forward/backward mạng 2 lớp — khớp PyTorch autograd (sai số $= 0$) và finite-difference (sai số $< 10^{-9}$)

Đại lượng	**Giá trị**
z_1	$(0.60,\ -0.40,\ 0.30)$
$a_1 = \text{ReLU}(z_1)$	$(0.60,\ 0.00,\ 0.30)$ — **neuron 2 "chết"** ($z_1[2] < 0$)
$\hat{y} = z_2$	0.29
L	0.25205
δ_2	-0.71
$\partial L/\partial W_2$	$(-0.426,\ 0.000,\ -0.213)^\mathsf{T}$
δ_1	$(-0.142,\ 0.000,\ -0.284)$
$\partial L/\partial W_1$	hàng 1 = hàng 2 = $(-0.142,\ 0.000,\ -0.284)$
$\partial L/\partial x$	$(-0.071,\ -0.071)$

Điểm sư phạm quan trọng. Cột thứ 2 của $\partial L/\partial W_1$ và $\partial L/\partial W_2$ đều bằng 0 — vì neuron 2 bị ReLU "khoá" ở forward ($z_1[2] = -0.4 < 0$) nên $\text{ReLU}'(z_1[2]) = 0$ chặn đứng toàn bộ gradient chảy qua neuron đó, bất kể δ phía sau lớn hay nhỏ. Đây là minh hoạ trực tiếp hiện tượng **"dead ReLU"**, sẽ nhắc lại ở Mục 1.4 và Mục 1.5.

1.3.2 Matrix Form (batch, khớp code NumPy/PyTorch)

Từ đây đổi quy ước: **hàng của X là một mẫu** — $X \in \mathbb{R}^{N \times n_0}$ với N = batch size (khớp `src/manual_nn.py` và mọi framework deep learning phổ biến):

$$Z^{(l)} = A^{(l-1)} W^{(l)} + b^{(l)} \in \mathbb{R}^{N \times n_l}, \qquad A^{(l)} = f(Z^{(l)}), \qquad A^{(0)} = X.$$

($b^{(l)} \in \mathbb{R}^{1 \times n_l}$ được **broadcast** — cộng vào mỗi trong N hàng như nhau; về mặt toán, tương đương nhân với vector cột toàn số 1: $\mathbf{1}_N\, b^{(l)}$.)

Backward (với $L = \frac{1}{N}\sum_i \text{loss}_i$ — trung bình theo batch, khớp `reduction='mean'` mặc định của PyTorch):

$$\frac{\partial L}{\partial Z^{(l)}} = \frac{\partial L}{\partial A^{(l)}} \odot f'(Z^{(l)}) \in \mathbb{R}^{N \times n_l}$$

$$\frac{\partial L}{\partial W^{(l)}} = \left(A^{(l-1)}\right)^\mathsf{T} \frac{\partial L}{\partial Z^{(l)}} \in \mathbb{R}^{n_{l-1} \times n_l}$$

$$\frac{\partial L}{\partial b^{(l)}} = \mathbf{1}_N^\mathsf{T} \frac{\partial L}{\partial Z^{(l)}} \in \mathbb{R}^{1 \times n_l} \quad \text{(tổng theo trục batch — hệ quả của broadcasting)}$$

$$\frac{\partial L}{\partial A^{(l-1)}} = \frac{\partial L}{\partial Z^{(l)}} \big(W^{(l)}\big)^\mathsf{T} \in \mathbb{R}^{N \times n_{l-1}}$$

Kiểm tra kích thước (lỗi hay gặp nhất khi code thủ công): $(n_{l-1} \times N)(N \times n_l) = n_{l-1} \times n_l$ khớp $W^{(l)}$; $(N \times n_l)(n_l \times n_{l-1}) = N \times n_{l-1}$ khớp $A^{(l-1)}$.

Quan hệ với Mục 1.3.1: cùng một phép tính, chỉ khác cách viết. Với $N = 1$ (bỏ batch): $Z_{\text{hàng}} = x^\mathsf{T} W + b \Leftrightarrow z_{\text{cột}} = W^\mathsf{T} x + b^\mathsf{T}$ — chuyển vị của nhau. Đây là lý do `src/manual_nn.py` (Phần A — Manual Backpropagation, xem Mục 2.2) triển khai đúng công thức batch ở trên, còn ví dụ bằng tay ở Mục 1.3.1 dùng quy ước cột cho gọn.

Ví dụ số đầy đủ dạng ma trận (batch $N = 2$, kiến trúc $3 \to 2 \to 1$, ReLU — toàn bộ tính bằng NumPy và đối chiếu PyTorch autograd khớp tuyệt đối trong `experiments/toy_example_verify.py::example_matrix_batch`):

$$X = \begin{pmatrix} 1.0 & 0.5 & -1.0 \\ 0.0 & 1.0 & 2.0 \end{pmatrix}, \quad W^{(1)} = \begin{pmatrix} 0.2 & -0.1 \\ 0.4 & 0.3 \\ -0.5 & 0.2 \end{pmatrix}, \quad b^{(1)} = \begin{pmatrix} 0.1 & -0.2 \end{pmatrix}$$

$$W^{(2)} = \begin{pmatrix} 0.6 \\ -0.3 \end{pmatrix}, \qquad b^{(2)} = \begin{pmatrix} 0.05 \end{pmatrix}, \qquad Y = \begin{pmatrix} 1.0 \\ 0.0 \end{pmatrix}.$$

Forward:

$$Z^{(1)} = XW^{(1)} + b^{(1)} = \begin{pmatrix} 1.00 & -0.35 \\ -0.50 & 0.50 \end{pmatrix}, \qquad A^{(1)} = \text{ReLU}(Z^{(1)}) = \begin{pmatrix} 1.00 & 0.00 \\ 0.00 & 0.50 \end{pmatrix}$$

$$\hat{Y} = A^{(1)} W^{(2)} + b^{(2)} = \begin{pmatrix} 0.65 \\ -0.10 \end{pmatrix}, \qquad L = \frac{1}{2N} \sum_i \|\hat{y}_i - y_i\|^2 = 0.033125.$$

Backward:

$$\frac{\partial L}{\partial Z^{(2)}} = \frac{\hat{Y} - Y}{N} = \begin{pmatrix} -0.175 \\ -0.050 \end{pmatrix}, \qquad \frac{\partial L}{\partial W^{(2)}} = \big(A^{(1)}\big)^\mathsf{T} \frac{\partial L}{\partial Z^{(2)}} = \begin{pmatrix} -0.175 \\ -0.025 \end{pmatrix}$$

$$\frac{\partial L}{\partial b^{(2)}} = \begin{pmatrix} -0.225 \end{pmatrix}, \qquad \frac{\partial L}{\partial A^{(1)}} = \frac{\partial L}{\partial Z^{(2)}} \big(W^{(2)}\big)^\mathsf{T} = \begin{pmatrix} -0.105 & 0.0525 \\ -0.030 & 0.0150 \end{pmatrix}$$

$$\frac{\partial L}{\partial Z^{(1)}} = \frac{\partial L}{\partial A^{(1)}} \odot \text{ReLU}'(Z^{(1)}) = \begin{pmatrix} -0.105 & 0.000 \\ 0.000 & 0.015 \end{pmatrix}$$

$$\frac{\partial L}{\partial W^{(1)}} = X^{\mathsf{T}} \frac{\partial L}{\partial Z^{(1)}} = \begin{pmatrix} -0.1050 & 0.000 \\ -0.0525 & 0.015 \\ 0.1050 & 0.030 \end{pmatrix}, \qquad \frac{\partial L}{\partial b^{(1)}} = \begin{pmatrix} -0.105 & 0.015 \end{pmatrix}$$

Lưu ý cột 2 của $\partial L/\partial Z^{(1)}$ có phần tử 0 ở hàng 1 (vì $Z^{(1)}_{1,2} = -0.35 < 0$ — neuron đó "chết" với mẫu 1) trong khi cột 1 có phần tử 0 ở hàng 2 (vì $Z^{(1)}_{2,1} = -0.50 < 0$ — neuron kia "chết" với mẫu 2 khác) — minh hoạ dead ReLU xảy ra *độc lập theo từng cặp (mẫu, neuron)*, không phải một hiện tượng toàn cục.

1.3.3 Backpropagation vs. Gradient Descent

Đây là hai khái niệm **thường bị nhầm là một**, nhưng đóng hai vai trò hoàn toàn khác nhau:

Bảng 1.4: Phân biệt Backpropagation và Gradient Descent

	Backpropagation	**Gradient Descent (và biến thể: SGD, Momentum, Adam,…)**
Là gì	Thuật toán **tính** $\nabla_\theta L$ hiệu quả bằng Chain Rule + computational graph	Quy tắc **cập nhật** tham số dùng gradient đã có: $\theta \leftarrow \theta - \eta \nabla_\theta L$
Đầu vào	Đồ thị tính toán + giá trị forward đã lưu	Gradient (từ bất kỳ nguồn nào — Backprop, finite-difference,…)
Đầu ra	Vector gradient $\nabla_\theta L$	Tham số mới θ_{t+1}
Có thể thay bằng gì	Về nguyên tắc: finite-difference (chậm, $O(P)$ forward) — cùng kết quả, khác chi phí	Về nguyên tắc: Momentum, RMSProp, Adam [3], Newton's method — cùng "nguyên liệu" gradient, khác cách dùng

Chốt lại: Backpropagation *không* "huấn luyện" mạng — nó chỉ trả lời *"loss đang thay đổi thế nào theo từng tham số?"*. Gradient Descent/optimizer mới là bước quyết định *"vậy nên sửa tham số ra sao?"*. Trong dự án này (Mục 2.1), toàn bộ 20 cấu hình thực nghiệm dùng *cùng một* Backpropagation (qua PyTorch autograd) và *cùng một* optimizer (SGD thuần) — biến duy nhất thay đổi là **initialization**, để cô lập đúng ảnh hưởng của θ_0.

1.4 Khởi tạo tham số (Parameter Initialization)

1.4.1 Vì sao Initialization quan trọng

Backpropagation cho biết *hướng* để sửa tham số, nhưng Gradient Descent chỉ di chuyển *cục bộ* quanh điểm xuất phát θ_0. Chọn θ_0 sai có thể khiến mạng đứng yên mãi mãi (Mục 1.4.2) hoặc gradient nổ/biến mất trước khi kịp học (Mục 1.5) — bất kể optimizer tốt đến đâu.

1.4.2 Zero Initialization

Nếu **mọi** trọng số của một lớp giống hệt nhau (kể cả $= 0$): với cùng input x, mọi neuron trong lớp có cùng $z_j = w_j \cdot x + b_j$ (nếu bias cũng khởi tạo giống nhau) $\Rightarrow$ cùng a_j. Ở backward, mọi neuron nhận cùng δ_j và cùng input $\Rightarrow$ cùng gradient $\Rightarrow$ sau cập nhật, trọng số của chúng **vẫn giống hệt nhau**. Quy nạp theo từng bước huấn luyện: các neuron trong lớp mãi mãi tính cùng một hàm — lớp có n_l neuron chỉ có "sức biểu diễn" như 1 neuron. Đây gọi là **mất khả năng phá vỡ đối xứng** (*symmetry breaking*) [2].

Với $W = 0$ cho *mọi* lớp (kể cả lớp output), phân tích còn mạnh hơn: $\partial L/\partial a^{(l-1)} = \delta^{(l)}(W^{(l)})^{\mathsf{T}} = \delta^{(l)} \cdot 0 = 0$ với mọi lớp ẩn — gradient của *mọi* lớp ẩn bằng **đúng bằng** 0, không chỉ "rất nhỏ". Chỉ lớp output (không có lớp nào phía sau để nhân với $W = 0$) còn nhận gradient khác 0 từ chính hàm loss. Mục 3.1 kiểm chứng đúng dự đoán này bằng số liệu thật (gradient đo được $= 0.0$ tuyệt đối tại mọi lớp ẩn khi khởi tạo Zero).

Ví dụ số truy vết 2 bước cập nhật (mạng $1 \to 2 \to 1$, activation linear để đơn giản, $x = 1,\ y = 1$, $\eta = 0.1$, mọi W, b khởi tạo 0):

Bảng 1.5: Truy vết 2 bước Gradient Descent với Zero-init — $W^{(1)}, W^{(2)}$ đứng yên tuyệt đối

Bước	$W^{(1)}$ (2 neuron)	$W^{(2)}$	$b^{(2)}$	L
Trước bước 1	$(0.0;\ 0.0)$	$(0.0;\ 0.0)$	0.00	0.500
Trước bước 2	$(0.0;\ 0.0)$	$(0.0;\ 0.0)$	0.10	0.405
Sau bước 2	$(0.0;\ 0.0)$	$(0.0;\ 0.0)$	0.19	—

Đúng như dự đoán: $W^{(1)}$ (2 trọng số vào 2 neuron ẩn) và $W^{(2)}$ **đứng yên tuyệt đối ở** 0 **qua cả 2 bước** ($dW^{(1)} = dW^{(2)} = 0$ luôn luôn) — hai neuron ẩn không bao giờ phân hoá. Toàn bộ việc "học" chỉ đến từ $b^{(2)}$ (bias lớp output, không bị vấn đề đối xứng vì chỉ có *một* neuron output) dịch chuyển dần $0 \to 0.1 \to 0.19$, kéo loss giảm nhẹ

$0.500 \to 0.405$ — mạng "học" được một hằng số (giá trị trung bình của y), không học được bất kỳ đặc trưng nào của x, khớp đúng dự đoán lý thuyết ở trên và số liệu thực nghiệm thật ở Mục 3.1.

1.4.3 Random Initialization

Khởi tạo ngẫu nhiên (vd. $W \sim \mathcal{N}(0, \sigma^2)$ với σ nhỏ cố định, không tính theo kích thước lớp) phá vỡ đối xứng — mỗi neuron xuất phát khác nhau nên học các đặc trưng khác nhau. Nhưng nếu σ chọn tuỳ tiện, mạng **sâu** vẫn có thể vanish/explode (Mục 1.5): đây là động lực cho các scheme "khoa học" hơn bên dưới, vốn chọn σ dựa trên kích thước lớp thay vì một hằng số cố định.

Vì sao không đơn giản chọn σ lớn hơn? Đây là câu hỏi tự nhiên tiếp theo — nếu σ quá nhỏ gây vanishing (Mục 3.1: Random với $\sigma = 0.01$), tại sao không chọn σ lớn hơn để bù? Vấn đề là **không có một σ cố định nào đúng cho mọi kiến trúc**: hệ số nhân dồn $r = n_{\text{in}}\sigma^2$ (Mục 1.4.4) phụ thuộc *cả* σ *lẫn* n_{in} — cùng một σ có thể vanishing ở lớp rộng (n_{in} lớn $\Rightarrow$ r lớn hơn dự kiến, có thể explode) và exploding ở lớp hẹp cạnh đó, hoặc ngược lại. **Ý tưởng cốt lõi** của LeCun/Xavier/He (Mục 1.4.4) chính là: thay vì đoán một hằng số, tính σ *lại từ đầu cho mỗi lớp* dựa theo n_{in} (và n_{out}) của chính lớp đó, để $r \approx 1$ **đồng thời ở mọi lớp** bất kể kiến trúc rộng/hẹp ra sao — đây là bước nhảy về chất so với "random đơn thuần", không chỉ là chọn số khác.

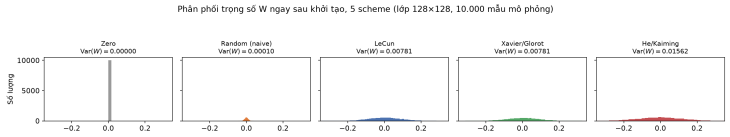
Giá trị mặc định trong framework thực tế: các thư viện Deep Learning phổ biến không dùng σ cố định kiểu "random ngây thơ" của báo cáo (vốn cố ý dùng làm đối chứng lý thuyết) mà mặc định sẵn một biến thể có scale theo n_{in} ngay từ đầu — `torch.nn.Linear` mặc định dùng phân phối Uniform theo biến thể Kaiming/He; Keras `Dense` mặc định dùng Glorot Uniform (cả hai không cần gọi hàm khởi tạo thủ công như Mục 2.2 nếu chấp nhận giá trị mặc định). Nói cách khác: **"Random (naive)" trong lưới thực nghiệm của báo cáo là trường hợp không ai dùng trong thực hành hiện đại** — được giữ lại có chủ đích như đường cơ sở lý thuyết (*baseline*) để làm nổi bật lý do các framework *không* dùng nó.

1.4.4 LeCun, Xavier/Glorot, He/Kaiming Initialization

Vì sao He nhân đôi so với LeCun? ReLU đặt khoảng một nửa output về 0 (phần $z < 0$), nên với z đối xứng quanh 0: $\text{Var}(\text{ReLU}(z)) \approx \frac{1}{2}\text{Var}(z)$ — để bù phần "mất" này, He et al. [6] nhân đôi variance của W so với công thức forward-preserving thuần ($1/n_{\text{in}}$) để

Bảng 1.6: Các scheme initialization dùng trong báo cáo (và trong lưới thực nghiệm Mục 3.1)

Scheme	**Công thức** $\mathrm{Var}(W)$	**Activation phù hợp**	**Nguồn**
Zero	0	Không dùng được	—
Random (naive)	σ^2 cố định, không theo n_{in}	Không khuyến nghị cho mạng sâu	thực hành phổ biến trước 2010
LeCun	$1/n_{\text{in}}$	Sigmoid/Tanh, input đã chuẩn hoá	LeCun et al. [4]
Xavier/Glorot	$2/(n_{\text{in}} + n_{\text{out}})$	Tanh/Sigmoid (activation đối xứng quanh 0)	Glorot and Bengio [5]
He/Kaiming	$2/n_{\text{in}}$	ReLU/Leaky ReLU	He et al. [6]



Hình 1.3: Phân phối $10\,000$ giá trị W mô phỏng ngay sau khởi tạo theo từng scheme (lớp 128×128, khớp $n_{\text{in}}{=}n_{\text{out}}{=}128$ của kiến trúc thực nghiệm Mục 2.1) — Zero co về một điểm; độ rộng phân phối tăng dần Random $<$ LeCun $=$ Xavier (khi $n_{\text{in}}{=}n_{\text{out}}$) $<$ He, đúng thứ tự $\mathrm{Var}(W)$ ở Bảng 1.6.

giữ $\mathrm{Var}(a) \approx \mathrm{Var}(x)$ qua mỗi lớp ReLU.

Hệ quả thứ hai của ReLU: lệch tâm dương (positive mean shift). Phân tích variance ở trên chỉ kiểm soát *bậc hai* (variance) của tín hiệu; ReLU còn tạo ra một hiệu ứng *bậc một* (kỳ vọng) mà công thức $\mathrm{Var}(W) = 2/n_{\text{in}}$ **không** khắc phục. Vì $\mathrm{ReLU}(z) = \max(0, z) \geq 0$ với mọi z, và với giả thiết chuẩn $z \sim \mathcal{N}(0, \sigma_z^2)$ (dùng xuyên suốt Mục 1.4.4):

$$\mathbb{E}[\mathrm{ReLU}(z)] = \int_0^\infty z \, \frac{1}{\sigma_z \sqrt{2\pi}} e^{-z^2/2\sigma_z^2} \, dz = \frac{\sigma_z}{\sqrt{2\pi}} \approx 0.40 \, \sigma_z \; > \; 0.$$

Nói cách khác, **đầu ra trung bình của mỗi neuron ReLU luôn dương**, bất kể trọng số W có kỳ vọng 0 (đối xứng) hay không — khác hẳn Tanh/Sigmoid-đã-chuẩn-hoá vốn có $\mathbb{E}[a] \approx 0$. Hệ quả: input của lớp kế tiếp không còn "căn giữa" quanh 0 như giả thiết ngầm trong mọi phân tích $\mathrm{Var}(z) = n_{\text{in}}\mathrm{Var}(W)\mathrm{Var}(x)$ ở Mục 1.4.4 (vốn coi x có kỳ vọng 0) — input không căn giữa là chính xác dạng vấn đề mà LeCun et al. [4] đã chỉ ra làm xấu điều kiện (*conditioning*) của bề mặt tối ưu và làm chậm hội tụ (phân tích gốc cho *input* của mạng, nhưng áp dụng y hệt cho input của *mọi lớp ẩn* khi lớp trước là ReLU). Qua nhiều lớp liên tiếp, lệch tâm dương ở mỗi lớp *cộng dồn* về mặt hiệu ứng lên độ xấu điều

kiện tổng thể, dù bản thân variance vẫn được He initialization giữ ổn định.

Vì sao có ELU, SELU. Lệch tâm dương của ReLU là động lực trực tiếp cho các activation "đã sửa tâm" ra đời sau đó: **ELU** (*Exponential Linear Unit*) — $\mathrm{ELU}(z) = z$ nếu $z > 0$, $\alpha(e^z - 1)$ nếu $z \leq 0$ — bão hoà về $-\alpha$ (thay vì cắt cứng về 0 như ReLU) ở nhánh âm, kéo $\mathbb{E}[a]$ về gần 0 hơn [7]. **SELU** (*Scaled ELU*) đẩy ý tưởng này xa hơn: chọn đúng hai hằng số λ, α sao cho phép lặp forward qua nhiều lớp tự động hội tụ kỳ vọng về 0 và variance về 1 — gọi là tính chất **tự chuẩn hoá** (*self-normalizing*) [8]. Kiến trúc thực nghiệm của báo cáo này (Mục 2.1) chỉ dùng Sigmoid/Tanh/ReLU/Leaky ReLU, không thử ELU/SELU — nêu ở đây để định vị đúng vị trí của He initialization trong dòng phát triển: giải quyết vấn đề *variance*, còn vấn đề *mean shift* được các kiến trúc activation sau đó giải quyết theo hướng khác.

Vì sao Xavier lấy trung bình n_{in} và n_{out}? Phân tích forward (giữ $\mathrm{Var}(z)$) cho $\mathrm{Var}(W) = 1/n_{\text{in}}$; phân tích backward (giữ $\mathrm{Var}(\partial L/\partial a)$ khi lan truyền ngược) cho $\mathrm{Var}(W) = 1/n_{\text{out}}$. Hai điều kiện này chỉ trùng nhau khi $n_{\text{in}} = n_{\text{out}}$ (hiếm khi đúng trong thực tế); Glorot and Bengio [5] chọn điểm dung hoà $\mathrm{Var}(W) = 2/(n_{\text{in}} + n_{\text{out}})$ để không hy sinh hẳn điều kiện nào.

Bias: cả 3 scheme trên đều khởi tạo $b = 0$ — bias không gây vấn đề đối xứng *giữa các neuron* theo cùng cách W gây ra (mỗi b_j vẫn có thể học giá trị khác nhau miễn W đã phá đối xứng), nên giữ $b = 0$ là điểm khởi đầu trung tính, đơn giản, được dùng thống nhất trong toàn bộ thực nghiệm của báo cáo (Mục 2.1).

Biến thể: bias dương nhỏ cho ReLU. Một số tài liệu thực hành (không dùng trong thực nghiệm của báo cáo này, nêu ở đây để đầy đủ) đề xuất khởi tạo b bằng một hằng số dương nhỏ, ví dụ $b = 0.01$, thay vì 0 cho các lớp dùng ReLU. Lý do: nếu $z = W^{\mathsf{T}}x + b < 0$ ngay ở bước forward đầu tiên (trước khi W kịp học), neuron đó rơi vào trạng thái $\mathrm{ReLU}(z) = 0$ và $\mathrm{ReLU}'(z) = 0$ — không có gradient chảy qua để cập nhật W, b, tức nguy cơ *dead ReLU* ngay từ lúc khởi tạo. Một độ lệch dương nhỏ trong b đẩy nhiều neuron hơn về phía $z > 0$ lúc khởi đầu, giảm xác suất "chết"sớm. Đánh đổi: $b > 0$ phá vỡ tính trung tính của bias và có thể làm lệch phân phối activation ban đầu; đây là lý do thực nghiệm của báo cáo giữ $b = 0$ để tách biệt hoàn toàn hiệu ứng của $\mathrm{Var}(W)$ (biến độc lập đang khảo sát) khỏi hiệu ứng phụ của bias.

1.5 Vanishing và Exploding Gradient

Theo Chain Rule, gradient tại lớp đầu là **tích** của nhiều đạo hàm cục bộ dọc theo L lớp:

$$\frac{\partial L}{\partial a^{(0)}} = \frac{\partial L}{\partial a^{(L)}} \prod_{l=1}^{L} \frac{\partial a^{(l)}}{\partial a^{(l-1)}}.$$

Nếu độ lớn trung bình mỗi nhân tử cục bộ $\approx g$:

- $g < 1$: tích $\approx g^L \to 0$ theo hàm mũ khi L tăng — **vanishing gradient**: lớp đầu gần như không nhận được tín hiệu học.
- $g > 1$: tích $\approx g^L \to \infty$ — **exploding gradient**: bước cập nhật quá lớn, loss dao động hoặc `NaN`.
- $g = 1$: tích ổn định ở mọi độ sâu — mục tiêu mà Xavier/He hướng tới (Mục 1.4.4).

Bảng 1.7: Minh hoạ số: hệ số nhân dồn g^L theo độ sâu L (đã tính lại bằng `experiments/toy_example_verify.py`)

g	$L = 1$	$L = 5$	$L = 10$	$L = 20$	$L = 50$
0.5 (vanishing)	5.0×10^{-1}	3.1×10^{-2}	9.8×10^{-4}	9.5×10^{-7}	8.9×10^{-16}
1.0 (ổn định)	1.0	1.0	1.0	1.0	1.0
2.0 (exploding)	2.0	3.2×10^{1}	1.0×10^{3}	1.0×10^{6}	1.1×10^{15}

Với $g = 0.5$, sau $L = 10$ lớp hệ số chỉ còn $\approx 1/1024$; sau $L = 50$ lớp gần như bằng 0 dưới độ chính xác số thực. Với $g = 2$, sau $L = 10$ lớp hệ số đã là 1024; sau $L = 50$ là hơn 10^{15}. Bảng 1.7 minh hoạ trực quan vì sao "giữ $g \approx 1$" (chính là mục tiêu của Xavier/He) quan trọng với mạng sâu — một sai lệch nhỏ ở g bị **khuếch đại theo hàm mũ** của số lớp.

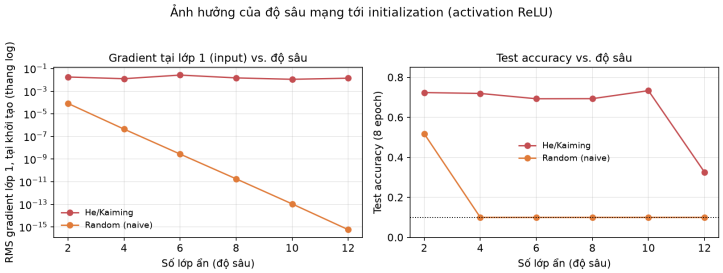
g phụ thuộc **cả ba yếu tố**: activation (hệ số co từ đạo hàm f' — vd. $\sigma'(z) \leq 0.25$ luôn luôn co lại), weight scale (qua $\mathrm{Var}(W)$, Mục 1.4.4), và độ sâu L (số lần nhân). Thực nghiệm Mục 3.1 (Hình **??**) đo trực tiếp gradient theo từng lớp trên mạng thật, xác nhận đúng ba yếu tố này cùng tác động.

Vì sao ReLU giúp giảm vanishing so với Sigmoid/Tanh. $\mathrm{ReLU}'(z) \in \{0, 1\}$ (tại $z = 0$, xem quy ước dưới đạo hàm Mục 1.1.4) — khi "sống" ($z > 0$), hệ số nhân đúng bằng 1 (không co lại như $\sigma'(z) \leq 0.25$); khi "chết" ($z \leq 0$), hệ số bằng 0 hoàn toàn (không "biến mất dần" mà "cắt đứt hẳn"). Đây là lý do ReLU không bị vanishing *do chính activation gây ra* như Sigmoid, nhưng đổi lại có rủi ro riêng: **dead ReLU** (neuron chết vĩnh viễn

nếu rơi vào vùng $z \leq 0$ và không bao giờ nhận gradient để "sống lại" — đã thấy trong ví dụ Bảng 1.3).

1.5.1 Thực nghiệm bổ sung: độ sâu ảnh hưởng thế nào tới initialization

Lưới 20 cấu hình chính (Mục 3.1) cố định độ sâu $= 6$ lớp ẩn. Để kiểm chứng trực tiếp dự đoán lý thuyết của mục này — *"vanishing/exploding trầm trọng hơn theo hàm mũ của độ sâu"* — nhóm chạy thêm một thực nghiệm riêng (`experiments/run_depth_experiment.py`): so sánh **He** và **Random (naive)**, activation ReLU, tại **6 độ sâu khác nhau** $L \in \{2, 4, 6, 8, 10, 12\}$ lớp ẩn (rộng cố định 128), đo (a) gradient RMS lớp 1 *ngay tại bước khởi tạo* (không cần huấn luyện, đo nhanh và tách bạch khỏi hiệu ứng optimizer) và (b) test accuracy sau 8 epoch huấn luyện thật (rút ngắn so với 15 epoch của lưới chính để tổng thời gian hợp lý với $6 \times 2 = 12$ cấu hình bổ sung).



Hình 1.4: Gradient RMS lớp 1 tại khởi tạo (trái) và test accuracy sau 8 epoch (phải), theo độ sâu mạng, activation ReLU. Đường chấm ở đồ thị phải = mức ngẫu nhiên (10%).

Ba quan sát chính (Hình 1.4):

1. **Random (naive) suy giảm gradient theo đúng đường thẳng trên thang log** khi độ sâu tăng tuyến tính — từ $\approx 8.3 \times 10^{-5}$ (L=2) xuống $\approx 5.7 \times 10^{-16}$ (L=12), tức **suy giảm 12 bậc độ lớn chỉ sau 10 lớp thêm vào** — xác nhận trực tiếp mô hình g^L (Mục 1.5, Bảng 1.7): trên thang log, $\log(g^L) = L \log g$ là một đường thẳng theo L, đúng như đồ thị cho thấy.
2. **Test accuracy của Random sụp xuống mức ngẫu nhiên ngay từ** L=4 — chỉ cần tăng độ sâu từ 2 lên 4 lớp là đủ để một initialization "tạm chấp nhận được" ở mạng nông (52% ở L=2, đã kém nhưng còn học được chút ít) trở thành hoàn toàn vô dụng.
3. **He giữ gradient ổn định (bậc** 10^{-2}**) ở mọi độ sâu từ 2 đến 12 lớp** — kể cả tại L=12, đúng như thiết kế của Mục 1.4.4: gradient RMS lớp 1 tại khởi tạo vẫn bình thường ($\approx 4 \times 10^{-2}$, Hình 1.4 trái, không hề suy giảm so với các độ sâu nhỏ hơn). Tuy nhiên, test accuracy tại L=12 **giảm mạnh xuống** 32.6%. Đối chiếu *cả hai*

loss (không chỉ accuracy) loại trừ ngay giả thuyết vanishing gradient: train loss $= 0.456$ (khớp fit tốt trên tập train, cùng bậc với mọi độ sâu khác — 0.36–0.42) trong khi test loss vọt lên 5.409 — **vượt xa** cả mức loss của một mô hình đoán ngẫu nhiên hoàn toàn ($-\ln(1/10) \approx 2.303$). Một mô hình *không học được gì* (vanishing gradient thực sự) sẽ có train loss *và* test loss đều mắc kẹt gần 2.3 như các cấu hình Zero/Random (Bảng 3.1); ở đây train loss thấp chứng tỏ mô hình *học được* trên tập train. Chữ ký số liệu này (train loss thấp, test loss cao hơn cả baseline ngẫu nhiên) là dấu hiệu kinh điển của **suy sụp năng lực khái quát hoá** (*generalization collapse* do overfitting), không phải vanishing gradient: một MLP 12 lớp ẩn ($283\,402$ tham số, tính theo đúng công thức Mục **??**) huấn luyện trên vỏn vẹn $5\,000$ mẫu, không regularization, dễ dàng ghi nhớ (*memorize*) nhiễu của tập train thay vì học đặc trưng tổng quát — và vì không có softmax/Cross-Entropy nào "giới hạn" độ tự tin sai, một dự đoán sai với xác suất gần 1 cho loss cực lớn (xem công thức Cross-Entropy Mục **??**: $L = -\log p_y$ tiến tới ∞ khi $p_y \to 0$).

Chương 2

THIẾT KẾ THỰC NGHIỆM

2.1 Phương pháp thực nghiệm

Mục tiêu: kiểm chứng bằng số liệu thật (không minh hoạ suông) ba dự đoán lý thuyết của Mục 1.4–1.5: (a) Zero/Random (naive) không học được; (b) initialization có scale đúng (LeCun/Xavier/He) học tốt hơn rõ rệt; (c) gradient theo từng lớp thể hiện đúng mẫu vanishing/exploding dự đoán bởi lý thuyết.

2.1.1 Kiến trúc mô hình

MLP 6 hidden layer, **cố định cho mọi cấu hình**, không BatchNorm/Dropout, không pretrained:

$$784 \to 128 \to 128 \to 128 \to 128 \to 128 \to 128 \to 10.$$

Lớp output luôn linear (logits), dùng `CrossEntropyLoss` (softmax nằm trong hàm loss). Độ sâu 6 lớp ẩn được chọn **cố ý đủ sâu** để chênh lệch giữa các initialization thể hiện rõ (một MLP 1–2 lớp gần như không nhạy với initialization, xem thảo luận Mục 3.2) mà vẫn huấn luyện nhanh trên CPU.

2.2 Hiện thực

Mã nguồn đầy đủ nằm trong `src/` (module hoá) và `experiments/` (script chạy) — xem cấu trúc trong `README.md`. Hai phần quan trọng nhất:

2.2.1 Part A: Manual Backpropagation

Part A — Manual Backpropagation (`src/manual_nn.py`): forward/backward tự viết bằng NumPy thuần, **không dùng autograd**, triển khai đúng công thức Mục 1.3.2:

Phạm vi có chủ đích của Part A, và ranh giới rõ ràng với Part B. `ManualMLP` dùng hàm mất mát **MSE** (không phải Cross-Entropy) và kiến trúc thu nhỏ tuỳ ý (vài chục neuron, xem các ví dụ số Mục 1.3.1–1.3.2) — đây là lựa chọn *có chủ đích*, không

phải giản lược tuỳ tiện: mục tiêu duy nhất của Part A là **kiểm chứng số học độc lập** (*toy numerical verification*) rằng công thức Chain Rule tự viết tay khớp chính xác với PyTorch autograd và finite-difference (Mục 2.2.2) trên những trường hợp đủ nhỏ để tính tay/kiểm tra bằng mắt được từng bước — **không** nhằm huấn luyện một mô hình thật. Việc huấn luyện thật, trên bài toán phân loại 10 lớp Fashion-MNIST với Softmax + Cross-Entropy (Mục **??**) và kiến trúc đủ sâu để initialization tạo khác biệt ($784 \to 128^{\times L} \to 10$, Mục 2.1), là công việc của **Part B** dưới đây — hai phần bổ trợ nhau chứ không thay thế nhau: Part A chứng minh *tính đúng đắn* của cơ chế backprop ở quy mô kiểm tra được; Part B áp dụng chính cơ chế đó (qua autograd, đã được Part A xác nhận là tương đương) ở quy mô đủ lớn để trả lời câu hỏi nghiên cứu chính của báo cáo.

2.2.2 Kiểm tra tính đúng đắn: Gradient Checking

Gradient Checking (`experiments/run_gradient_check.py`): đối chiếu **ba** cách tính gradient trên cùng một mạng, cùng một batch: (1) manual (NumPy), (2) PyTorch autograd (xây một bản sao dùng đúng trọng số của mạng manual, để so sánh công bằng), (3) finite-difference $\big(f(w+h) - f(w-h)\big)/(2h)$, $h = 10^{-6}$.

Bảng 2.1: Kết quả Gradient Checking (mạng $6 \to 8 \to 5 \to 1$, activation Tanh)

Lớp	Kích thước W	max|manual−autograd|	max rel. error (vs. finite-diff)
0	6×8	4.39×10^{-9}	3.90×10^{-9}
1	8×5	6.35×10^{-9}	7.60×10^{-9}
2	5×1	4.71×10^{-9}	3.37×10^{-9}

Sai số tối đa ($\approx 10^{-9}$) nằm ở mức sai số làm tròn số thực (`float64`), **không phải sai số thuật toán** — kết quả **PASS** với ngưỡng chấp nhận 10^{-4} (log đầy đủ: `results/logs/gradient_check_report.json`). Gradient checking hữu ích chính vì nó **cô lập lỗi**: nếu `manual` sai (sai chiều ma trận, quên chia batch, sai dấu…) trong khi `autograd` và `finite-diff` vẫn khớp nhau, ta biết ngay lỗi nằm ở khâu tính gradient thủ công — không cần debug toàn bộ vòng lặp huấn luyện.

Chương 3

KẾT QUẢ VÀ THẢO LUẬN

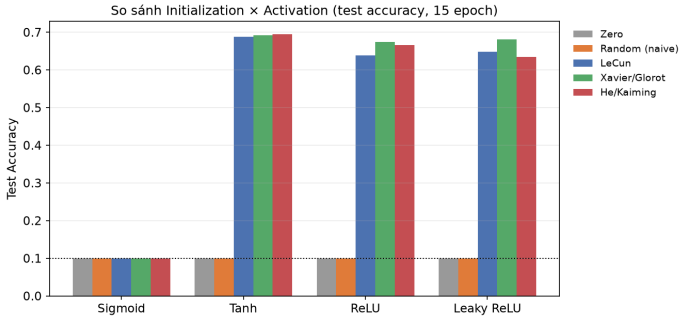
3.1 Kết quả thực nghiệm

Toàn bộ số liệu mục này lấy trực tiếp từ `results/tables/initialization_comparison.csv` và `results/logs/run_*.json` (20 file, sinh bởi `experiments/run_initialization.py`, tổng thời gian chạy ≈ 1 phút CPU) — không có số liệu minh hoạ hay ước lượng bằng tay.

3.1.1 Bức tranh tổng thể: test accuracy

Bảng 3.1: Test accuracy sau 15 epoch, toàn bộ 20 cấu hình

	Sigmoid	Tanh	ReLU	Leaky ReLU
Zero	0.100	0.100	0.100	0.100
Random (naive)	0.100	0.100	0.100	0.100
LeCun	0.100	0.689	0.638	0.648
Xavier/Glorot	0.100	0.692	0.674	0.682
He/Kaiming	0.100	0.694	0.667	0.634



Hình 3.1: Test accuracy, toàn bộ lưới 5×4 (đường chấm = mức ngẫu nhiên 10%).

Ba quan sát chính (Hình 3.1, Bảng 3.1):

1. **Zero và Random (naive) không bao giờ vượt mức ngẫu nhiên (**10%**)**, với *mọi* activation — khớp dự đoán lý thuyết Mục 1.4.2 (Zero: gradient lớp ẩn $\equiv 0$) và Mục 1.4.3 (Random không scale theo n_{in} nên vẫn vanishing ở độ sâu 6 lớp).

2. **Sigmoid thất bại với MỌI initialization**, kể cả He/Xavier vốn "đúng lý thuyết" cho các activation khác. Mục 3.2 phân tích nguyên nhân bằng chính số liệu gradient đo được.
3. **LeCun/Xavier/He đều hoạt động tốt với Tanh/ReLU/Leaky ReLU** (64–69%), chênh lệch giữa 3 scheme này nhỏ hơn nhiều so với chênh lệch Zero/Random vs. có-scale — khớp việc cả ba đều bảo toàn variance tín hiệu đúng bậc-độ-lớn, chỉ khác hệ số nhân (1 so với 2).

3.2 Thảo luận

3.2.1 Vì sao Sigmoid thất bại ngay cả với He/Xavier?

Đây là kết quả đáng chú ý nhất của thực nghiệm (Bảng 3.1): lý thuyết Mục 1.4.4 nói He/Xavier "bảo toàn variance tín hiệu", nhưng điều đó chỉ đúng về *weight scale* — nó không thể bù cho việc $\sigma'(z) \leq 0.25$ **luôn luôn đúng**, bất kể variance của z là bao nhiêu. Với 6 lớp, hệ số suy giảm trần do riêng activation gây ra đã là $0.25^6 \approx 2.4 \times 10^{-4}$ — cộng thêm 15 epoch với SGD thuần ($\eta = 0.05$, không momentum) là không đủ để vượt qua mức suy giảm này trong thời gian huấn luyện ngắn của báo cáo. Số liệu Mục 3.1 xác nhận đúng cơ chế này: gradient RMS lớp 1 của Sigmoid+He chỉ $\approx 1.6 \times 10^{-5}$ (so với $\approx 2.8 \times 10^{-2}$ của ReLU+He) — khác 0 (không "chết" như Zero-init) nhưng quá nhỏ để tạo ra bước cập nhật đáng kể ở LR đang dùng.

Nguyên nhân thứ hai, độc lập với trần đạo hàm: lệch tâm của Sigmoid gây dao động dích dắc (zig-zagging). $\sigma'(z) \leq 0.25$ giải thích tín hiệu *yếu*; nhưng ngay cả khi tín hiệu đủ mạnh, Sigmoid còn một vấn đề thứ hai làm SGD hội tụ chậm: $\sigma(z) \in (0, 1)$ **không đối xứng quanh** 0 ($\mathbb{E}[\sigma(z)] \approx 0.5 > 0$, cùng bản chất lệch tâm dương đã phân tích cho ReLU ở Mục 1.4.4, chỉ khác Sigmoid lệch tâm nhiều hơn hẳn vì miền giá trị bị nén hoàn toàn vào $(0, 1)$). Theo công thức Mục 1.3.2, $\partial L/\partial W^{(l)} = (A^{(l-1)})^\mathsf{T} \delta^{(l)}$, tức với *một* neuron output cố định j, dấu của $\partial L/\partial w_{ij}$ với mọi i chỉ phụ thuộc dấu của δ_j — vì $A_i^{(l-1)} > 0$ với **mọi** i (Sigmoid luôn dương). Hệ quả: **mọi trọng số đi vào cùng một neuron buộc phải tăng đồng loạt hoặc giảm đồng loạt ở mỗi bước cập nhật** — không thể di chuyển tự do theo từng hướng riêng như khi input đã căn giữa quanh 0. Về mặt hình học, quỹ đạo Gradient Descent trong không gian trọng số bị ép đi theo đường **dích dắc** (*zig-zagging*) thay vì đường thẳng ngắn nhất tới cực tiểu — đúng hiện tượng mà LeCun et al. [4] khuyến nghị tránh bằng cách *căn giữa input* (*centering*) trước khi đưa vào mạng; ở đây vấn đề tái xuất hiện ngay giữa các lớp ẩn vì Sigmoid tự làm

mất tính căn giữa mà chính lớp trước đã có. Cơ chế này giải thích thêm phần nào vì sao Sigmoid+He/Xavier hội tụ chậm *ngay cả khi* gradient chưa suy giảm tới mức triệt tiêu hoàn toàn.

3.3 Hạn chế

Các hạn chế dưới đây phần lớn là **lựa chọn có chủ đích** để giữ thực nghiệm gọn, chạy nhanh trên CPU và giữ đúng nguyên tắc fair experiment (Mục 2.1), không phải sai sót. Mục đích liệt kê rõ ràng ở đây là để người đọc biết chính xác *phạm vi* mà các kết luận của báo cáo (Mục 3.1–3.2) có thể áp dụng, và phạm vi nào cần thêm thực nghiệm mới có thể khẳng định.

- **Quy mô nhỏ**: subset $5\,000/1\,000/2\,000$ mẫu, 15 epoch, để chạy nhanh trên CPU — không phản ánh kết quả ở quy mô full Fashion-MNIST hay nhiều epoch hơn (đặc biệt với Sigmoid, có thể cần rất nhiều epoch hơn để vượt qua giai đoạn gradient nhỏ ban đầu).
- **Một seed duy nhất** mỗi cấu hình — không đo được variance giữa nhiều lần chạy độc lập, nên các chênh lệch nhỏ (vd. He vs. Xavier trên ReLU, Mục 3.2) không thể khẳng định có ý nghĩa thống kê.
- **SGD thuần, không tinh chỉnh LR riêng** cho từng cấu hình (cố ý, để giữ fair experiment) — nghĩa là một số cấu hình (vd. Sigmoid) có thể học nhanh hơn nếu được tinh chỉnh LR riêng; kết luận "Sigmoid thất bại" cần hiểu trong đúng phạm vi thiết lập này.
- **Không BatchNorm/Dropout/LR schedule** — cố ý loại bỏ để initialization là biến duy nhất, nhưng vì vậy kết quả không đại diện cho thiết lập huấn luyện hiện đại (nơi BatchNorm thường giảm đáng kể độ nhạy với initialization).
- **Validation dao động mạnh** (Hình **??**) do tập val nhỏ ($1\,000$ mẫu) — nên đọc xu hướng chung thay vì giá trị từng epoch riêng lẻ.
- **Lớp output cũng khởi tạo theo cùng scheme** với các lớp ẩn (xem `src/models.py`) dù lớp output không có activation phi tuyến phía sau — lựa chọn đơn giản hoá, có thể không tối ưu tuyệt đối nhưng không ảnh hưởng tới các kết luận định tính của báo cáo.

KẾT LUẬN

Báo cáo trình bày Backpropagation như hệ quả trực tiếp của Chain Rule áp dụng có hệ thống trên một computational graph (Mục 1.1.3–1.3), phân biệt rõ với Gradient Descent (Mục 1.3.3): Backprop *tính* gradient, Gradient Descent *dùng* gradient đó để cập nhật tham số. Về Initialization, báo cáo chứng minh bằng cả suy luận toán học (Mục 1.4) lẫn thực nghiệm thật (Mục 3.1) rằng lựa chọn θ_0 ảnh hưởng quyết định tới việc mạng có học được hay không: Zero-init xoá hoàn toàn khả năng phá vỡ đối xứng (gradient lớp ẩn $\equiv 0$, đo được chính xác bằng 0.0 trong thực nghiệm); Random (naive) và ngay cả He/Xavier với activation bão hoà sâu (Sigmoid, 6 lớp) vẫn có thể vanishing đủ mạnh để chặn đứng việc học trong thời gian huấn luyện ngắn. Hai chủ đề tưởng như tách biệt của báo cáo hoá ra là hai mặt của cùng một câu hỏi: *làm sao gradient "chảy" xuyên suốt một mạng nhiều lớp mà không biến mất hay bùng nổ* — Backpropagation là cơ chế *tính* dòng chảy đó, Initialization là điều kiện *ban đầu* quyết định dòng chảy đó có sống sót qua nhiều lớp hay không.

Đóng góp cụ thể của báo cáo (tổng hợp lại các mục thực nghiệm): (1) triển khai và kiểm chứng Backpropagation thủ công bằng NumPy, khớp tuyệt đối PyTorch autograd và finite-difference tới sai số 10^{-9} (Mục 2.2.2); (2) lưới thực nghiệm đầy đủ $5 \times 4 = 20$ cấu hình initialization$\times$activation trên Fashion-MNIST, xác nhận trực tiếp các dự đoán lý thuyết bằng số liệu đo được, không suy diễn (Mục 3.1); (3) thực nghiệm bổ sung về độ sâu mạng (Mục 1.5.1), cho thấy cả hiện tượng vanishing kinh điển (Random, suy giảm 12 bậc độ lớn qua 12 lớp) lẫn một giới hạn ít được nhắc tới của chính He initialization ở độ sâu rất lớn.

Hướng mở rộng nếu có thêm thời gian/tài nguyên (liệt kê đầy đủ ở Mục 3.3): lặp lại lưới thực nghiệm với nhiều seed để có khoảng tin cậy; thêm Batch Normalization như một biến thực nghiệm thứ ba để đo trực tiếp mức độ nó làm giảm độ nhạy với initialization (dự đoán ở Mục 3.2 nhưng chưa kiểm chứng thực nghiệm); mở rộng thực nghiệm độ sâu (Mục 1.5.1) sang kiến trúc có residual connection để so sánh trực tiếp với MLP thuần.

Tài liệu tham khảo

[1] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.

[2] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.

[3] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *International Conference on Learning Representations (ICLR)*, 2015, arXiv:1412.6980.

[4] Y. LeCun, L. Bottou, G. B. Orr, and K.-R. Müller, "Efficient backprop," in *Neural Networks: Tricks of the Trade*, 2nd ed., ser. Lecture Notes in Computer Science. Springer, 2012, vol. 7700, bản gốc xuất bản trong LNCS vol. 1524, 1998.

[5] X. Glorot and Y. Bengio, "Understanding the difficulty of training deep feedforward neural networks," in *Proceedings of the 13th International Conference on Artificial Intelligence and Statistics (AISTATS)*, vol. 9. PMLR, 2010, pp. 249–256.

[6] K. He, X. Zhang, S. Ren, and J. Sun, "Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2015, arXiv:1502.01852.

[7] D.-A. Clevert, T. Unterthiner, and S. Hochreiter, "Fast and accurate deep network learning by exponential linear units (ELUs)," in *International Conference on Learning Representations (ICLR)*, 2016, arXiv:1511.07289.

[8] G. Klambauer, T. Unterthiner, A. Mayr, and S. Hochreiter, "Self-normalizing neural networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017, arXiv:1706.02515.